

ОПЕРАЦИИ IOCTL

Нижний Новгород, 2025г.

ДРАЙВЕР КАК ПОСРЕДНИК

Проблема: Как приложение может взаимодействовать с "железом"?

Решение: Драйвер предоставляет унифицированный интерфейс через **файловую систему**

Концепция: "Всё есть файл" (Everything is a file).

- Устройство представляется в системе как файл (/dev/my_device).
- Приложение использует стандартные системные вызовы: open(), read(), write(), ioctl(), close().
- Эти вызовы попадают в соответствующие функции драйвера.

ДРАЙВЕР КАК ПОСРЕДНИК

Символьные устройства (Character Devices)

Доступ к данным последовательный, побайтовый.

Примеры: клавиатура, мышь, последовательный порт, принтер.

Файл в /dev/: c в начале прав доступа (crw--w----).

Блочные устройства (Block Devices)

Доступ к данным блочный, возможен в любом порядке.

Примеры: жёсткие диски, SSD, USB-флешки.

Файл в /dev/: b в начале прав доступа (brw-rw----).

Имеют собственную подсистему с кэшированием и планированием запросов.

Сетевые устройства (Network Devices)

Не представлены в виде файлов в /dev/.

Используют специализированный сетевой интерфейс (сокеты).

СТРУКТУРА FILE_OPERATIONS

- **Назначение:** Это "таблица виртуальных функций" драйвера. Ядро ищет в ней адреса функций, которые нужно вызвать при системном вызове.
- **Определение:** Набор указателей на функции. Драйвер инициализирует только те, которые он поддерживает.

```
struct file_operations {  
    struct module *owner;  
    loff_t (*llseek) (struct file *, loff_t, int);  
    ssize_t (*read) (struct file *, char __user *, size_t, loff_t *);  
    ssize_t (*write) (struct file *, const char __user *, size_t, loff_t *);  
    long (*unlocked_ioctl) (struct file *, unsigned int, unsigned long);  
    int (*open) (struct inode *, struct file *);  
    int (*release) (struct inode *, struct file *);  
    // ... и многие другие  
};
```

ПРИМЕР РЕГИСТРАЦИИ FILE_OPERATIONS

```
static struct file_operations my_fops = {  
    .owner = THIS_MODULE,  
    .open = my_device_open,  
    .release = my_device_release,  
    .read = my_device_read,  
    .write = my_device_write,  
    .unlocked_ioctl = my_device_ioctl,  
};
```

```
static int __init my_init(void){  
    int major;  
    major = register_chrdev(0, "my_device", &my_fops); // Динамический номер  
    // ... создание /dev/my_device  
    return 0;  
}
```

ЧТО ТАКОЕ IOCTL?

Input/Output Control - универсальный механизм для передачи управляющих команд драйверу, которые не вписываются в модель read/write.

Использование: Настройка параметров устройства, получение статуса, переключение режимов работы.

Пример: установка скорости последовательного порта, сброс устройства, запрос версии драйвера.

IOCTL С ТОЧКИ ЗРЕНИЯ ПРИЛОЖЕНИЯ

```
int ioctl(int fd, unsigned long request, ... /* arg */);
```

- fd: Файловый дескриптор, полученный из open().
- request: Код команды. Определяется драйвером.
- arg: Дополнительный аргумент (указатель, число). Передаётся "как есть".

IOCTL С ТОЧКИ ЗРЕНИЯ ПРИЛОЖЕНИЯ

```
int fd = open("/dev/my_device", O_RDWR);
int baud_rate = 115200;

// Установка скорости (команда SET_BAUD, передаём указатель на
значение)
if (ioctl(fd, MY_DEVICE_SET_BAUD, &baud_rate) == -1) {
    perror("ioctl failed");
}

int status;
if (ioctl(fd, MY_DEVICE_GET_STATUS, &status) == -1) {
    perror("ioctl failed");
}

close(fd);
```

ПРОТОТИП IOCTL В ДРАЙВЕРЕ

`long (*unlocked_ioctl) (struct file *filp, unsigned int cmd, unsigned long arg);`

- Не использует Big Kernel Lock (BKL), что улучшает производительность.
- Используется в современных драйверах
- **Параллелизм** - несколько процессов могут одновременно вызывать ioctl

Типичные сценарии использования:

- **Сетевые устройства** - настройка IP, MAC-адресов
- **Аудиоустройства** - регулировка громкости, выбор источника
- **Датчики** - калибровка, настройка точности
- **Дисплеи** - яркость, контрастность, разрешение
- **Системные устройства** - перезагрузка, сброс настроек

СОЗДАНИЕ IOCTL-КОМАНД

Проблема: Коды команд должны быть уникальными, чтобы не было конфликтов между драйверами.

Макросы: (определены в `<linux/ioctl.h>`)

- `_IO(type, nr)`: Команда без передачи данных.
 - `_IOR(type, nr, datatype)`: Чтение данных из драйвера (Read).
 - `_IOW(type, nr, datatype)`: Запись данных в драйвер (Write).
 - `_IOWR(type, nr, datatype)`: Двухнаправленная передача данных (Read/Write)
-
- `type`: 8-битный "магический" номер, уникальный для драйвера (часто буква).
 - `nr`: Номер команды (8 бит).
 - `datatype`: Тип данных, который передаётся (например, `int`).

СОЗДАНИЕ ЮСТЛ-КОМАНД

// Команды с проверкой размера данных

```
#define MY_DEV_GET_STATUS  _IOR(MY_DEVICE_TYPE, 1, struct dev_status)
```

```
#define MY_DEV_SET_CONFIG  _IOW(MY_DEVICE_TYPE, 2, struct dev_config)
```

// Без данных

```
#define MY_DEV_RESET      _IO(MY_DEVICE_TYPE, 3)
```

```
#define MY_DEV_READ_DATA  _IOWR(MY_DEVICE_TYPE, 4, struct dev_data)
```

СОЗДАНИЕ ИОCTL-КОМАНД

```
long my_device_ioctl(struct file *filp, unsigned int cmd, unsigned long arg)
{
    struct my_device_data *data = filp->private_data;
    int status, new_baud;
    struct config_data config;

    // Проверяем, что команда корректна для нашего устройства
    if (_IOC_TYPE(cmd) != MY_DEVICE_IOC_MAGIC)
        return -ENOTTY; // "Not a typewriter" - стандартная ошибка

    if (_IOC_NR(cmd) > MY_DEVICE_IOC_MAXNR)
        return -ENOTTY;
```

СОЗДАНИЕ ЮСТЛ-КОМАНД

```
switch (cmd) {  
    case MY_DEVICE_GET_STATUS:  
        status = get_device_status();  
        // Копируем результат в пользовательское пространство  
        if (copy_to_user((int __user *)arg, &status, sizeof(status)))  
            return -EFAULT;  
        break;  
  
    case MY_DEVICE_SET_BAUD:  
        // Копируем аргумент ИЗ пользовательского пространства  
        if (copy_from_user(&new_baud, (int __user *)arg, sizeof(new_baud)))  
            return -EFAULT;  
        set_device_baud_rate(new_baud);  
        break;
```

СОЗДАНИЕ ЮСТЛ-КОМАНД

```
case MY_DEVICE_RESET:
```

```
    reset_device();
```

```
    break;
```

```
case MY_DEVICE_READ_CONFIG:
```

```
    // Чтение конфигурации в kernel space
```

```
    get_device_config(&config);
```

```
    // Возврат обновлённой конфигурации в user space
```

```
    if (copy_to_user((struct config_data __user *)arg, &config, sizeof(config)))
```

```
        return -EFAULT;
```

```
    break;
```

```
default:
```

```
    return -ENOTTY; // Неизвестная команда
```

```
}
```

```
return 0; // Успех
```

```
}
```

КОДЫ ВОЗВРАТА И ОБРАБОТКА ОШИБОК

Общее правило: Возвращайте 0 при успехе, отрицательное значение при ошибке.

Стандартные коды ошибок ядра (`<linux/errno.h>`):

- EINVAL – Неверный аргумент.
- ENOMEM – Недостаточно памяти.
- EFAULT – Ошибка при доступе к пользовательскому указателю.
- ENOTTY – Неверная или неподдерживаемая команда ioctl.
- EBUSY – Устройство занято.
- EAGAIN – Операция должна быть повторена

Важно: Всегда проверяйте возвращаемые значения `copy_to_user/copy_from_user`.